

OOPJ Day 6

Sat, 12 Sept 26

Recap: Classpath and User Libraries

- JAR files (Java Application Archives) are zip files containing compiled classes
 - Added via the classpath variable or a user-defined library variable in Eclipse
 - Can modify the JRE classpath to include custom JARs, but only JARs, not raw class folders
 - Scanner class is not a user-added library; it is part of the JRE system library by default
 - Lives inside `java.base` → `java.util` → Scanner
 - All packages and classes visible in a new Java project come from the JRE system library
 - Linking another project's bin folder via Build Path → Add Class Folder makes its classes available
 - The dependency project must remain open in Eclipse
 - Closing it causes a build path error: "project is missing required library"
 - Analogy: trying to turn on a camera that isn't connected; the dependency simply isn't there
-

Upcasting and Downcasting

- Upcasting: assigning a derived object to a base class reference (implicit, no cast syntax needed)
 - Syntax: `Base obj = new Derived();`
 - Goes up the hierarchy from derived → base, hence "upcasting"
 - Why use it: generalization; one reference can point to any object in the hierarchy
 - Example: `Base obj = new Derived()` or `Base obj = new SpecialDerived()` using the same reference
 - A `show()` function taking a Base parameter can handle Derived and SpecialDerived without overloading
 - Limitation: through a base reference, only base class members (`n1`, `n2`, `display`) are accessible
 - Members unique to the derived type (e.g., `n6` in `SpecialDerived`) are hidden
- Downcasting: converting a base reference back to a specific derived type (explicit cast required)
 - Syntax: `SpecialDerived xobj = (SpecialDerived) obj;`
 - Goes down the hierarchy from base → derived, hence "downcasting"

- Required when you need to access derived-specific members after generalization
 - Must be done explicitly because the compiler cannot guarantee the object's actual type at compile time
 - Key rule: a base class reference can point to a derived object because the derived object always contains the full base part
-

Pass by Value vs. Call by Reference

- Java does not support true pass-by-reference (that is a C++ feature using the & syntax)
 - In C++, `int& x = value` creates an alias; no new memory is allocated; same symbol table entry
 - In Java, passing an object always allocates a new reference variable that copies the memory address
 - What actually happens when passing an object in Java:
 - `obj` holds memory address 600 (pointing to the heap object)
 - A new reference `pobj` is created at a different memory address (e.g., 400) and receives the value 600
 - Both `obj` and `pobj` point to the same heap object, so changes to the object's fields are reflected
 - But reassigning `pobj = new Base()` only changes `pobj`'s local copy; `obj` is unaffected
 - Passing a primitive (e.g., `int value = 99`):
 - A new variable `pValue` is created and the value 99 is copied into it
 - Changes to `pValue` inside the method do not affect the original variable
 - Correct terminology:
 - Passing an object reference → technically pass by value (the address is the value being copied)
 - Informally called "call by reference" because you are calling the object by its reference
 - True pass by reference (C++ style) is not possible in Java
-

Static vs. Dynamic Polymorphism

- Static polymorphism (compile-time / early binding):
 - The compiler decides which function to call at compile time based on the pointer/reference type
 - In C++, without the virtual keyword: `Base* ptr = new Derived(); ptr->display();` always calls `Base::display`

- Compiler replaces the call with a direct instruction: `Base::display()` regardless of the actual object
 - Reason: at compile time, the compiler only knows the type of ptr (Base), not what it points to at runtime
 - If the object is created conditionally (e.g., based on user input), the compiler cannot predict the type
 - Dynamic polymorphism (runtime / late binding):
 - The function to call is resolved at runtime based on the actual object type
 - Triggered in C++ by declaring the base method as virtual
 - In Java, all non-final, non-static methods are virtual by default; no virtual keyword exists
 - Enables true function overriding across the inheritance hierarchy
-

Virtual Function Table (VFT) Internals

- Three changes occur when a class contains at least one virtual function:
 - A hidden private data member called `__vfptr` (virtual function table pointer) is added to the class
 - Size: 4 bytes (it is a pointer)
 - Every object of that class carries this pointer
 - A virtual table (VFT) is created once per class (not per object)
 - Contains memory addresses of all virtual functions in that class only
 - Non-virtual functions (e.g., `void add()`) are not included
 - Example: if Base has virtual `show()` at index 0 and virtual `display()` at index 1, the VFT stores their addresses at offsets 0×4 and 1×4
 - Every virtual function call is replaced by the compiler with a runtime formula:
 - $\text{ptr} \rightarrow \text{__vfptr} + (\text{offset of method} \times 4)$
 - The `__vfptr` resolves to the VFT of the actual object at runtime
- Derived class VFT construction:
 - Compiler creates a separate VFT for the derived class
 - For each virtual method in the base: if not overridden in derived, the base class method address is copied in
 - If overridden in derived, the derived class method address replaces the base entry
 - Any new virtual methods added in derived are appended at the bottom
 - The `__vfptr` is inherited automatically into the derived class
- Runtime resolution example:
 - `ptr = new Base()` → `__vfptr` points to Base VFT (e.g., at address 1000) → display at 1004 → `Base::display` fires
 - `ptr = new Derived()` → `__vfptr` points to Derived VFT (e.g., at address 2000) → display at 2004 → `Derived::display` fires

- The formula is fixed at compile time; only the VFT address (1000 or 2000) is filled in at runtime
 - Java equivalence:
 - Same VFT mechanism applies internally in Java
 - All methods are virtual by default; no virtual keyword needed
 - Cannot inspect the VFT directly in Java (unlike C++ with Visual Studio debugger)
 - Memory size note (observed in Visual Studio debug):
 - Adding one virtual function increases object size by 4 bytes (the `__vfptr`)
 - Adding more virtual functions does not increase size further; only one pointer per object
 - On 64-bit architecture, memory padding may cause sizes to appear as 8 or 16 bytes (holes in classes / memory padding for performance)
-

Function Overriding, Overloading, and Hiding

- Function overriding:
 - Requires an inheritance relationship between two classes
 - The derived class defines a method with exactly the same name and signature as the parent
 - Changes the behavior of the parent method for objects of the derived type
 - In C++: parent method must be declared virtual for true overriding; otherwise it is function hiding
 - In Java: overriding happens by default since all methods are virtual
 - Function hiding (C++ without virtual):
 - Same method name in base and derived, but base method is not virtual
 - Compiler resolves the call based on the pointer type, not the object type
 - Not true overriding; the base version is always called through a base pointer
 - Function overloading:
 - Same method name with different number, type, or sequence of arguments
 - Must be within the same scope (same class)
 - Two methods with the same name in base and derived are not overloading; they are in different scopes
 - `@Override` annotation in Java:
 - Optional but strongly recommended
 - Tells the compiler: "I intend to override a method from the parent class"
 - Compiler checks that a matching signature exists in the parent; if not, compilation error
 - Prevents typo errors (e.g., `Display` vs. `display` being treated as a new method silently)
 - Does not affect runtime behavior; overriding works without it in Java
-

Abstract Classes, final Keyword, and Key Rules

- Abstract method:
 - A method with no body, declared using the abstract keyword
 - Equivalent to a pure virtual function in C++ (virtual void show() = 0;)
 - Forces every concrete derived class to provide an implementation
 - Syntax: public abstract void show(); — no curly braces allowed
 - Abstract class:
 - Any class containing at least one abstract method must be declared abstract
 - Cannot be instantiated directly (new Base()) → compilation error: cannot instantiate abstract type)
 - Can contain both abstract and non-abstract (concrete) methods
 - Can contain final methods (non-overridable concrete methods are valid inside an abstract class)
 - Cannot contain an abstract method that is also final (contradictory: must override vs. cannot override)
 - A final class cannot contain abstract methods (cannot inherit to override)
 - final keyword — three distinct uses:
 - final variable (constant):
 - Value cannot be changed after initialization
 - Must be initialized at one of: point of declaration, initialization block, or constructor
 - Cannot initialize in more than one of these locations
 - Syntax: protected final int num1 = 1;
 - final method:
 - Cannot be overridden by any derived class
 - Syntax: public final void show() { ... }
 - Derived class attempting to override gets: “cannot override final method from Base”
 - final class:
 - Cannot be subclassed (inherited from)
 - Syntax: public final class Base { ... }
 - Any extends attempt gives: “the type Derived cannot subclass the final type Base”
 - super keyword for constructor chaining:
 - Used inside a derived constructor to explicitly call a specific base class constructor
 - Syntax: super(name, address, age, gender, basicSalary);
 - Must be the first statement in the derived constructor — no exceptions
 - If not specified, Java implicitly calls the base default constructor
 - If the base class has only a parameterized constructor (no default), and super() is not called explicitly, compilation error occurs
-

Constructors in Inheritance

- When a derived object is created, the base constructor always executes first, then the derived constructor
 - Reason: the derived object contains the base part; the base must be constructed before the derived additions
 - Analogy: a parent must exist before a child can exist
 - Object type (not reference type) determines which constructors fire:
 - Base obj = new Derived() → Base constructor fires, then Derived constructor fires
 - The reference type (Base) is irrelevant to constructor invocation
 - Default constructor behavior:
 - If no constructor is defined, the compiler provides a default constructor automatically
 - If any constructor (default or parameterized) is explicitly defined, the compiler no longer provides a default
 - Derived class default constructor implicitly calls super() (base default constructor)
 - If base has no default constructor, derived must explicitly call super(args) or compilation fails
 - Parameterized constructor in base class:
 - Derived must use super(args) as the first statement to invoke the correct base constructor
 - Multiple derived constructors can call different base constructors via super() overloading
-

Assignment: Employee Menu-Driven Program (Due Monday)

- Goal: build a menu-driven console application to manage employees using OOP principles
- Class hierarchy design:
 1. Employee (abstract class): common fields for all employee types
 - protected String name
 - protected String address
 - protected int age
 - protected boolean/char gender
 - protected float basicSalary
 - Getters and setters for all fields with validation (e.g., age < 18 or age > 65 → set default 21)
 - Parameterized constructor taking all five fields (so subclasses use super() to initialize them)
 2. Manager extends Employee: adds protected float hra
 3. Engineer extends Employee: adds protected float overtime

4. SalesPerson extends Employee: adds protected float commission
 5. Each subclass constructor calls super(name, address, age, gender, basicSalary) as first statement, then sets its own field
 - Storage: use fixed-size arrays of object references for each type
 1. Syntax: Manager[] arrManager = new Manager[N];
 2. Array creation only allocates reference slots (all null initially); each slot needs new Manager(...) separately
 3. Syntax: arrManager[0] = new Manager(name, address, age, gender, basicSalary, hra);
 - Menu options to implement by Monday:
 1. Add: sub-menu → 1. Manager / 2. Engineer / 3. SalesPerson / 4. Exit (back to main menu)
 - Accept all required fields via ConsoleInput, create object, store in respective array
 2. Display: print details of all added employees (managers, engineers, salespersons)
 3. Sort by name:
 - Sub-options: A. Ascending / B. Descending / C. Exit
 - Use String.compareTo(): returns 0 (equal), -1 (first < second), 1 (first > second)
 - Implement manually using bubble sort or similar; no built-in sort methods allowed
 4. Exit: terminate the program
 - Additional inheritance exercises (separate from the assignment) also due Monday; estimated ~1 hour
 - Use only concepts taught so far in Java; no built-in Collections or Arrays.sort()
 - Array size: pick any fixed value; no dynamic resizing required
-

Chat with meeting transcript:

<https://notes.granola.ai/t/bcebf5e3-20fa-4e8a-ad24-2191ef6996b1-00demib2>